



**CURSO DE GRADUAÇÃO TECNOLÓGICA EM ANÁLISE E DESENVOLVIMENTO  
DE SISTEMAS**

CARLOS DANIEL DA SILVA  
ERIC DOUGLAS RODRIGUES SOARES  
GABRIEL SANTANA LABIAPARI  
GUILHERME FERNANDO DE SOUZA MACIEL  
VINICIUS PASSOS PEIXOTO

**PROJETO INTEGRADOR MÓDULO 1: TESTE DE SOFTWARE  
FLAPPY BIRD**

**Campus Metrô Adolfo Pinheiro – São Paulo**

**2024**

Carlos Daniel da Silva  
Eric Douglas Rodrigues Soares  
Gabriel Santana Labiapari  
Guilherme Fernando de Souza Maciel  
Vinicius Passos Peixoto

**PROJETO INTEGRADOR MÓDULO 1: TESTE DE SOFTWARE**  
**Flappy Bird**

Trabalho de Projeto Integrador como conclusão  
do módulo 1 do Curso de Graduação em  
Análise e Desenvolvimento de Sistemas.

Orientador: Prof. Glauco Antonio do  
Nascimento

**CAMPUS METRÔ ADOLFO PINHEIRO – SÃO PAULO**  
**2024**

## RESUMO

O presente trabalho tratará sobre o tema de teste de software, utilizando como estudo de caso o jogo *Flappy Bird*, criado através do Scratch, linguagem de programação desenvolvida pelo Instituto de Tecnologia de Massachussetts (MIT). Neste trabalho, foi feita uma extensa análise sobre os recursos presentes dentro do jogo, evidenciando os seus pontos positivos e negativos. A metodologia utilizada inclui uma pesquisa sobre a importância da etapa de testes no processo de desenvolvimento de qualquer software. Através da análise do software, conclui-se que o jogo apresenta alguns pontos de melhoria, os quais podem agregar para que a experiência se torne ainda melhor para todos os jogadores. Este projeto visa contribuir com o aprofundamento da discussão sobre a importância dos testes de software, oferecendo uma visão fundamentada sobre o tema.

**Palavras-chave:** Teste de software; Projeto Integrador; *Flappy Bird*.

## LISTA DE FIGURAS

|                                                                               |    |
|-------------------------------------------------------------------------------|----|
| Figura 1 – Tela de menu principal .....                                       | 9  |
| Figura 2 - Início de um novo jogo .....                                       | 10 |
| Figura 3 - Movimentação do personagem e registro de pontuação .....           | 10 |
| Figura 4 - Tela de fim de jogo em caso de colisão #1 .....                    | 11 |
| Figura 5 - Tela de fim de jogo em caso de colisão #2 .....                    | 12 |
| Figura 6 - Tela de fim de jogo em caso de colisão #3 .....                    | 12 |
| Figura 7 - Elemento “chão” dentro da ferramenta de edição .....               | 14 |
| Figura 8 - Algoritmo presente no elemento "chão" .....                        | 14 |
| Figura 9 - Montanhas utilizadas no plano de fundo .....                       | 15 |
| Figura 10 - Algoritmo de movimentação presente nas montanhas .....            | 15 |
| Figura 11 - Elemento "título do jogo" dentro da ferramenta de edição .....    | 16 |
| Figura 12 - Elemento "botão de início" dentro da ferramenta de edição .....   | 17 |
| Figura 13 - Algoritmo presente no elemento "botão de início" .....            | 17 |
| Figura 14 - Personagem e suas variações (à esquerda).....                     | 18 |
| Figura 15 - Algoritmo responsável pela animação de voo.....                   | 19 |
| Figura 16 - Algoritmo que interrompe operações .....                          | 19 |
| Figura 17 – Demonstração em jogo da <i>hitbox</i> visível .....               | 20 |
| Figura 18 - Algoritmo responsável por simular a gravidade do personagem ..... | 20 |
| Figura 19 - Algoritmo responsável por simular a gravidade do personagem ..... | 21 |
| Figura 20 - Algoritmo de registro de pontuação e colisão com obstáculos ..... | 21 |
| Figura 21 - Algoritmo responsável pelo voo do personagem .....                | 22 |
| Figura 22 - Cano e suas variações (à esquerda) .....                          | 23 |
| Figura 23 - Demonstração em jogo das linhas vermelhas .....                   | 23 |
| Figura 24 - Algoritmo responsável por clonar obstáculos.....                  | 24 |
| Figura 25 - Algoritmo de variação de obstáculos .....                         | 24 |
| Figura 26 - Algoritmo presente nas linhas vermelhas .....                     | 25 |
| Figura 27 - Contador de pontuação e suas variações (à esquerda).....          | 25 |
| Figura 28 - Algoritmo presente no contador de pontuação .....                 | 26 |

|                                                                                       |    |
|---------------------------------------------------------------------------------------|----|
| Figura 29 - Algoritmo presente no contador de pontuação .....                         | 26 |
| Figura 30 - Tela de fim de jogo dentro da ferramenta de edição.....                   | 27 |
| Figura 31 - Botão para reiniciar um novo jogo.....                                    | 27 |
| Figura 32 - Botão para retornar ao menu principal.....                                | 28 |
| Figura 33 - Algoritmo presente na tela de fim de jogo.....                            | 28 |
| Figura 34 - Algoritmo presente na tela de fim de jogo.....                            | 28 |
| Figura 35 – Algoritmo responsável por alternar sprites (botão de reiniciar jogo)..... | 29 |
| Figura 36 – Algoritmo responsável por iniciar um novo jogo .....                      | 29 |
| Figura 37 - Algoritmo responsável por alternar sprites (botão de menu principal) .... | 30 |
| Figura 38 – Algoritmo responsável por retornar ao menu principal .....                | 30 |
| Figura 39 – Parte do algoritmo, originalmente com valor 10 variável "GRAVITY!" ....   | 31 |
| Figura 40 - Resultado ao reduzir o valor da variável "GRAVITY!" para 9.....           | 31 |
| Figura 41 – Primeira evidência.....                                                   | 32 |
| Figura 42 – Segunda evidência.....                                                    | 32 |

## SUMÁRIO

|                                         |    |
|-----------------------------------------|----|
| 1 INTRODUÇÃO .....                      | 7  |
| 2 CONCEITOS INICIAIS SOBRE O JOGO ..... | 8  |
| 3 TESTE DE CAIXA PRETA.....             | 9  |
| 4 TESTE DE CAIXA BRANCA .....           | 13 |
| 4.1 CHÃO .....                          | 13 |
| 4.2 PLANO DE FUNDO.....                 | 14 |
| 4.3 TÍTULO DO JOGO .....                | 16 |
| 4.4 BOTÃO DE INÍCIO .....               | 16 |
| 4.5 PERSONAGEM .....                    | 18 |
| 4.6 HITBOX .....                        | 20 |
| 4.7 CANOS .....                         | 22 |
| 4.8 CONTADOR DE PONTUAÇÃO .....         | 25 |
| 4.9 TELA DE FIM DE JOGO .....           | 27 |
| 5 CONCLUSÃO .....                       | 33 |
| 6 CRONOGRAMA .....                      | 34 |
| REFERÊNCIAS.....                        | 35 |

## 1 INTRODUÇÃO

No processo de desenvolvimento de qualquer software, a etapa de teste de software é de extrema importância, pois é esta que garantirá que os requisitos determinados pelo cliente serão atendidos. Segundo Souza e Gasparotto (2013, p.3), “[...] além de colaborar para obtenção de um produto com qualidade, a atividade de teste tem o papel fundamental na redução de custos com possíveis reparos ao sistema”.

Ao pensarmos sobre a indústria de entretenimento de jogos, o teste de software desempenha um papel crucial, garantindo a entrega do jogo com a qualidade esperada pelos consumidores finais, livrando-os de *bugs* que atrapalhem o desempenho e progressão dos jogadores nos jogos.

Neste cenário, como estudo de caso para este trabalho, selecionamos uma releitura do jogo *Flappy Bird* desenvolvida por Travister88 através da linguagem de programação Scratch. Temos como objetivo principal a análise geral do jogo, de todas as suas operações e ações, bem como a análise da estrutura do código-fonte, a fim de entender como cada elemento funciona e como cada elemento do jogo se conecta.

Muito além de apenas avaliar a jogabilidade, buscamos formas de contribuir para a melhoria na qualidade do software, procurando corrigir falhas de desenvolvimento que possam prejudicar a experiência do jogador; para isso, efetuamos diversos testes no ambiente do jogo, a fim de verificar se o desempenho é conforme o esperado e como o software se comporta na prática.

## 2 CONCEITOS INICIAIS SOBRE O JOGO

O jogo *Flappy Bird* tem suas origens datadas de 2013, época em que o programador Nguyễn Hà Đông o desenvolveu para distribuição nas lojas de aplicativos de plataformas mobile como iOS e Android, alcançando o seu ápice no início de 2014.

A releitura criada por Travister88 através do site Scratch reutiliza da mecânica do jogo original, porém, com design dos elementos do jogo de sua própria autoria. No jogo, controlamos o voo de um pássaro amarelo através do clique com o botão esquerdo do mouse, botão de barra de espaço ou pelo toque na tela, caso o usuário esteja jogando em uma plataforma mobile. O objetivo principal do jogo é fazer com que o personagem desvie dos obstáculos que surgem nas partes superior e inferior do cenário, representados por canos verdes.

À medida que o personagem avança por entre os canos, um contador que registra o número de obstáculos ultrapassados aumenta. Caso o personagem colida com algum dos canos, ou ainda com o chão ou a borda superior do cenário, o jogador perde o jogo.

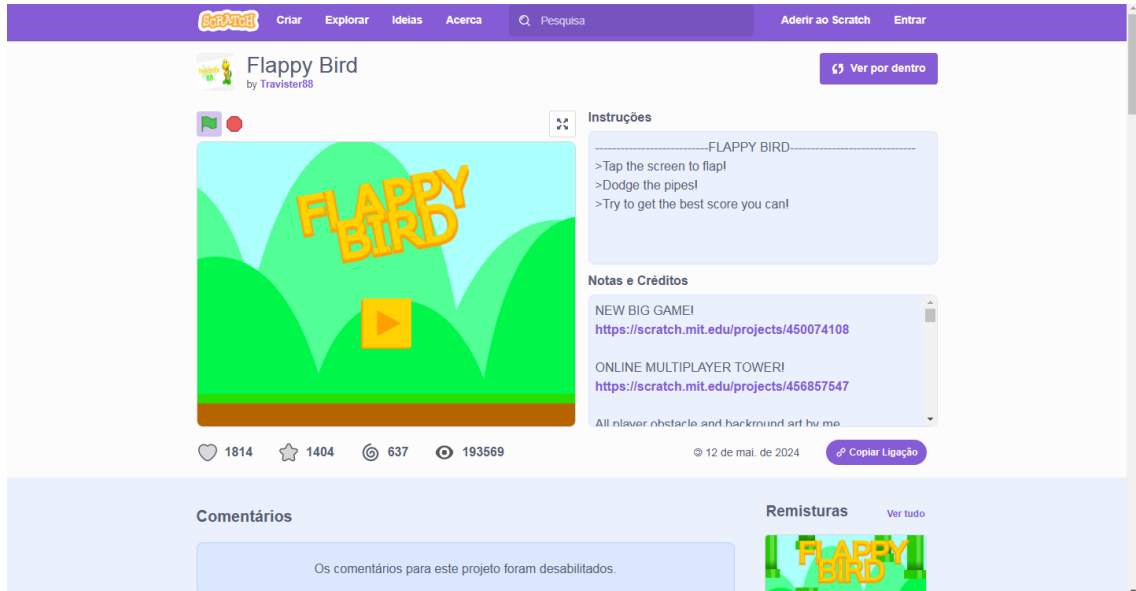


### 3 TESTE DE CAIXA PRETA

Quando pensamos no processo de teste de softwares, existem duas possibilidades para a execução, sendo estas: os testes de caixa preta e caixa branca. O teste de caixa preta consiste na verificação das operações e ações de uma aplicação, ou seja, testar a funcionalidade do software em si. É uma etapa mais simples, visto que não é necessário ter um conhecimento técnico sobre o código do software. No contexto do jogo objeto deste trabalho, o teste de caixa preta auxiliará no processo de análise da jogabilidade, das funcionalidades de cada comando dentro do jogo e da interação do personagem com o cenário.

Ao entrarmos na página do jogo no site do Scratch, somos recebidos pelo menu principal (figura 1), contendo o título do jogo e um botão interativo utilizado para iniciar um novo jogo. Ao clicarmos no botão citado, um novo jogo é iniciado, evidenciando que este cumpre com a função esperada.

**Figura 1** – Tela de menu principal

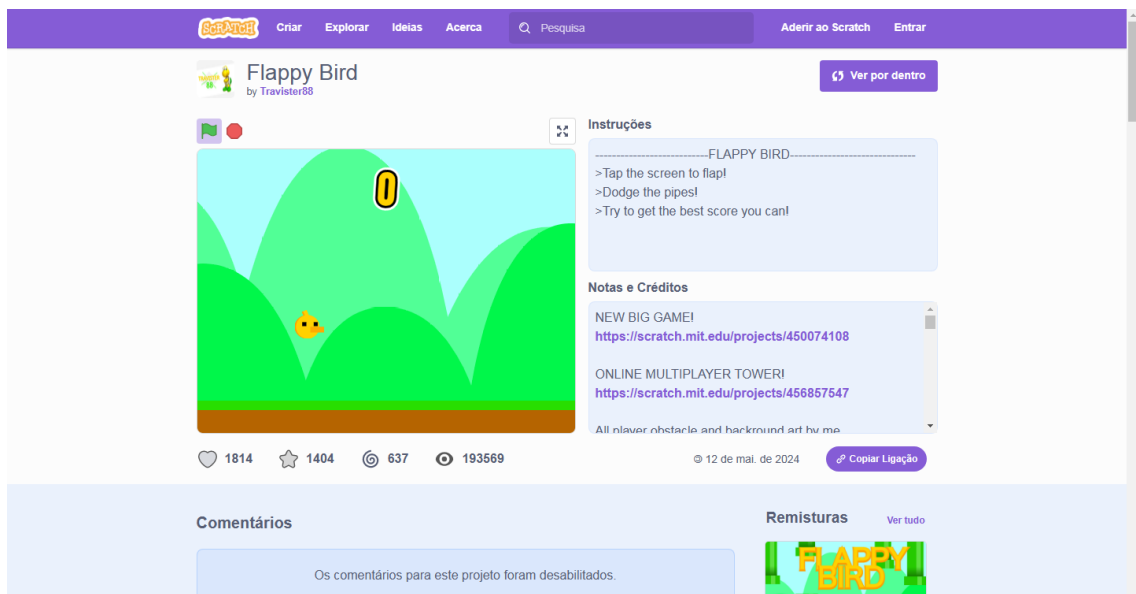


Fonte: Captura de tela pelos autores, 2024.

Após isso, surgem na tela os principais elementos do jogo, como o cenário, o personagem controlado pelo jogador e o contador de obstáculos ultrapassados (figura 2). Foi possível identificar que, ao acionarmos o comando responsável por fazer o personagem voar, este funcionou conforme o esperado. Ademais, conforme

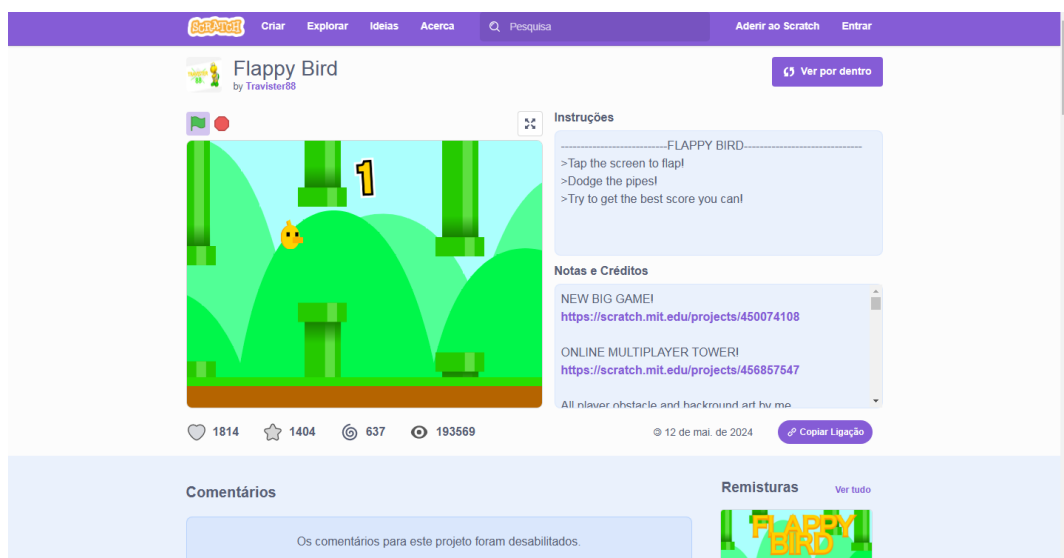
os obstáculos surgem no cenário, foi possível evidenciar também que, ao ultrapassá-los, o contador registrou que o jogador ultrapassou um obstáculo, conforme o esperado (figura 3).

**Figura 2 - Início de um novo jogo**



Fonte: Captura de tela pelos autores, 2024.

**Figura 3 - Movimentação do personagem e registro de pontuação**



Fonte: Captura de tela pelos autores, 2024.

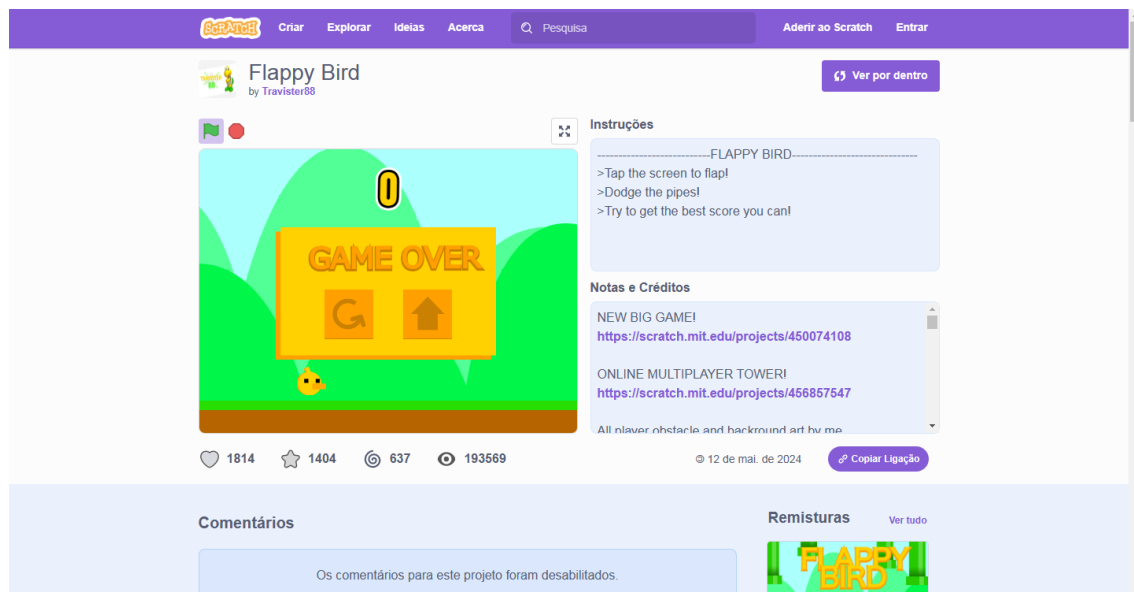
Testadas as operações e ações anteriormente citadas, partimos para o teste de colisão do personagem com os obstáculos. Inicialmente, evidenciamos que, caso

o personagem colida com o chão do cenário, o jogador perde o jogo conforme o esperado (figura 4). Entretanto, identificamos que o personagem acaba por não tocar totalmente o chão, fato este que será retomado mais a frente em nosso trabalho.

Logo após, identificamos que, caso o personagem colida com a borda superior da tela, o jogador perde o jogo (figura 5). Por fim, caso o personagem colida com um dos canos que surgem no cenário, o jogador também perde o jogo, conforme o esperado (figura 6).

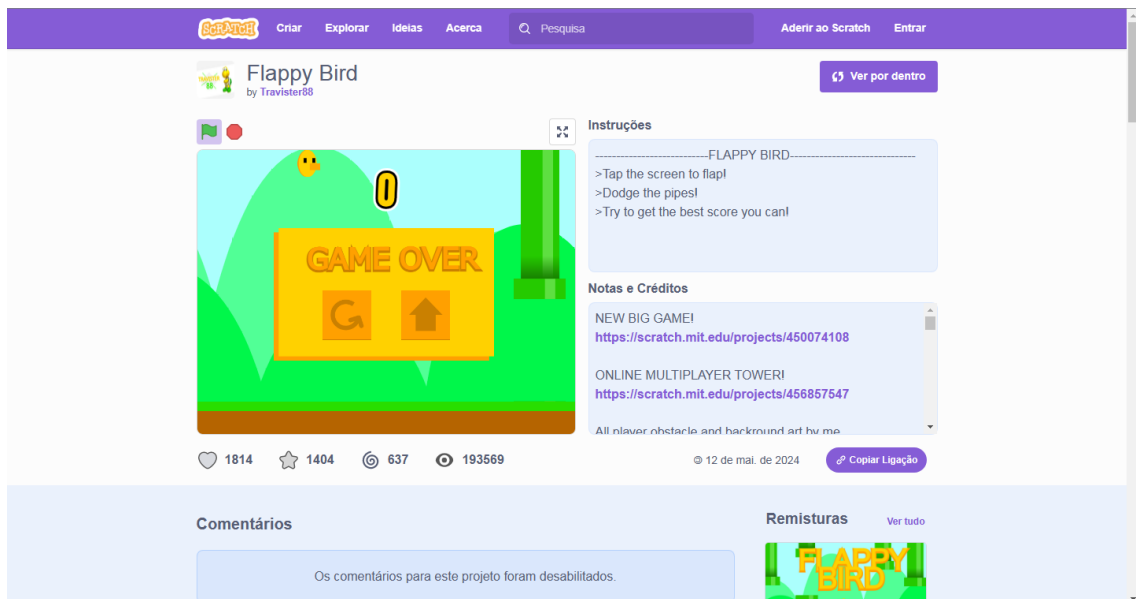
A ação de perder o jogo resulta na interrupção e surgimento da tela de fim de jogo, a qual possui dois botões interativos. Ao clicarmos no primeiro botão, o jogador pode recomeçar o jogo. Ao clicarmos no segundo, o jogador será redirecionado ao menu principal do jogo; ambos os botões mencionados estão funcionando perfeitamente, conforme o esperado.

**Figura 4** - Tela de fim de jogo ao colidir com o chão do cenário



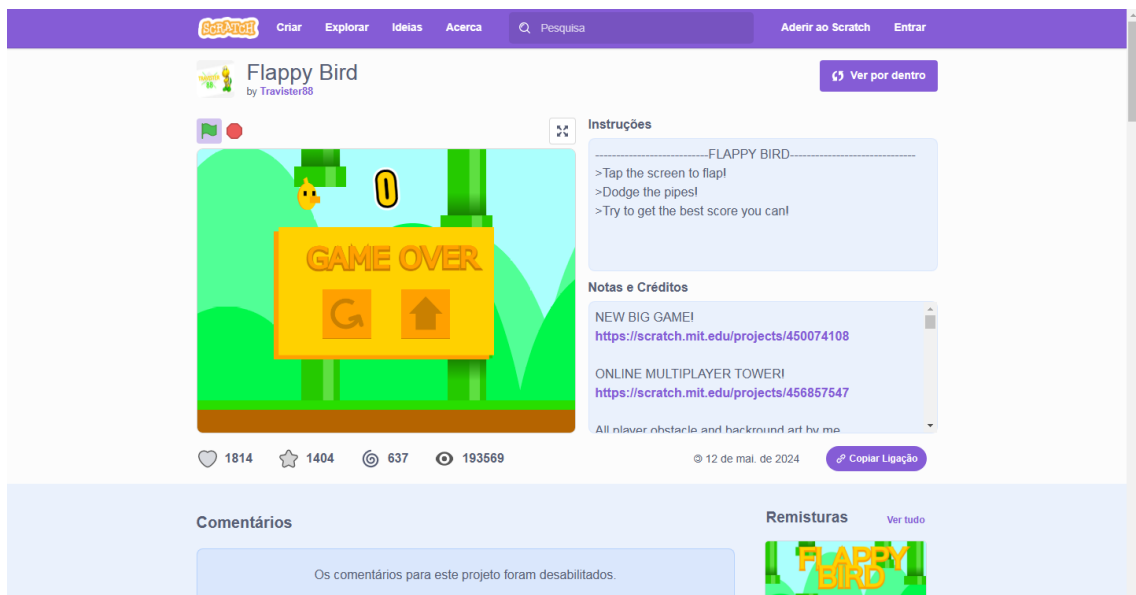
Fonte: Captura de tela pelos autores, 2024.

**Figura 5** - Tela de fim de jogo ao colidir com a borda superior do cenário



Fonte: Captura de tela pelos autores, 2024.

**Figura 6** - Tela de fim de jogo ao colidir com um cano



Fonte: Captura de tela pelos autores, 2024.

No que tange o teste de caixa preta, podemos determinar os seguintes pontos como possíveis sugestões de melhorias:

1. Em diversos momentos da jogabilidade, o jogo apresenta alguns problemas de otimização, ocorrendo travamentos que ocasionam na colisão do jogador com os obstáculos, atrapalhando o objetivo principal do jogo.

2. Ao pensarmos no cenário do jogo, pode se observar que o plano de fundo é composto apenas por montanhas verdes, as quais ocupam um espaço que poderia servir para incluir mais elementos como nuvens, árvores, cenários urbanos, etc.
3. Considerando a natureza e elementos *Arcade* do jogo, a adição de um quadro com o registro de pontuações dos jogadores seria uma ótima inclusão. Caso este fosse integrado com a conta do jogador no Scratch, isso contribuiria para a promoção de um cenário competitivo entre os jogadores e, conseqüentemente, aumentaria a popularidade do jogo.

#### **4 TESTE DE CAIXA BRANCA**

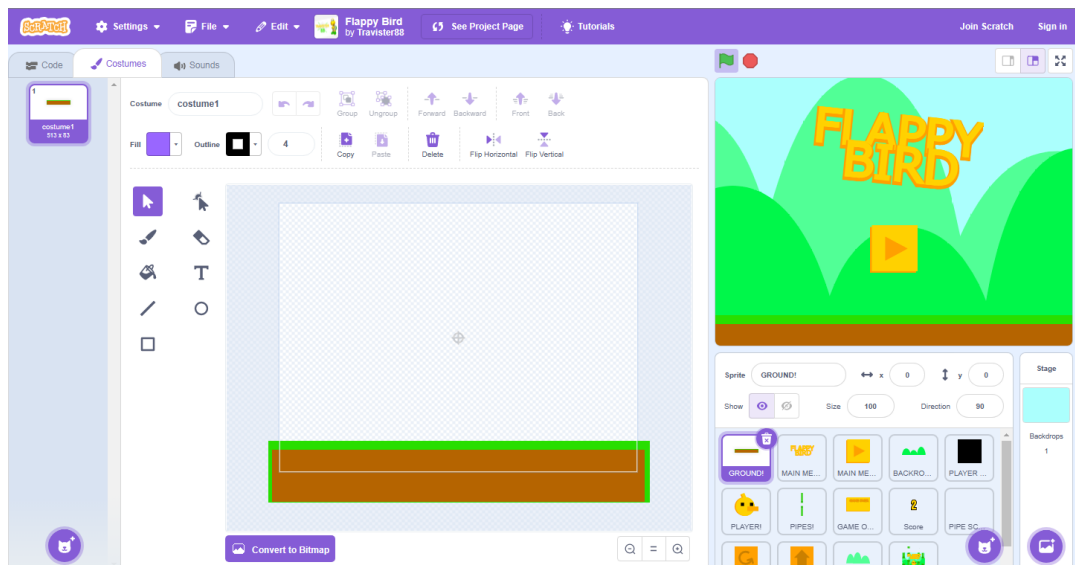
Ao pensarmos nesta etapa, é correto afirmar que o teste de caixa branca é baseado no código interno da aplicação. Ou seja, ao efetuar o teste de software, o código-fonte do software é analisado, levando em consideração todo o algoritmo e elementos presentes neste.

No contexto do objeto deste trabalho, fizemos uma análise a fundo do código-fonte do jogo e toda a lógica de programação aplicada aos elementos que o compõem, sendo que, qualquer pessoa pode o fazer através da ferramenta de edição do código-fonte, disponibilizada pelo próprio site do Scratch.

##### **4.1 CHÃO**

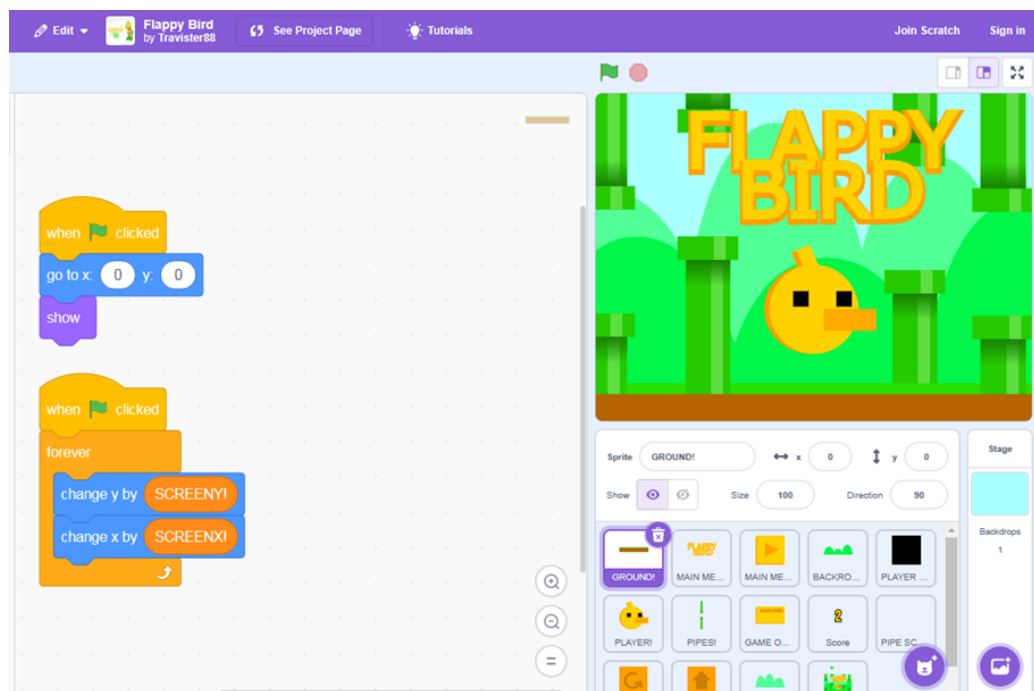
Neste jogo, o chão é um elemento estático, servindo como um complemento ao cenário. Entretanto, vale destacar que o chão também tem um papel crucial na lógica de colisão do personagem (figura 20), conforme anteriormente citado.

**Figura 7 - Elemento “chão” dentro da ferramenta de edição**



Fonte: Captura de tela pelos autores, 2024.

**Figura 8 - Algoritmo presente no elemento "chão"**



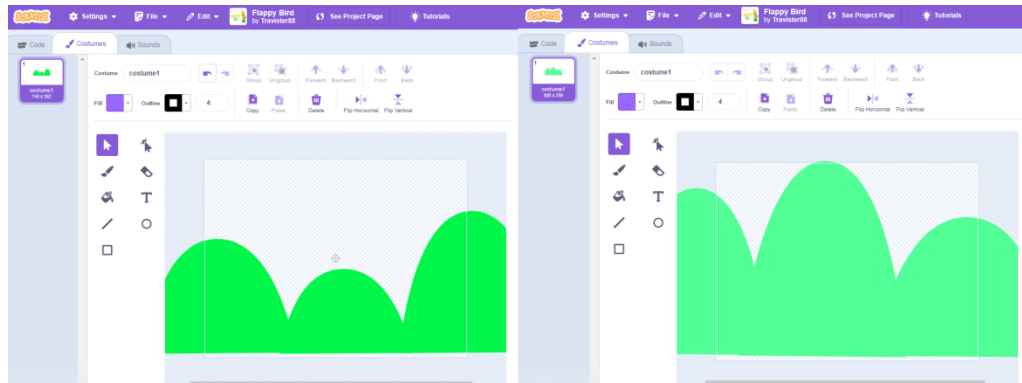
Fonte: Captura de tela pelos autores, 2024.

## 4.2 PLANO DE FUNDO

O plano de fundo é composto por um estágio de fundo na cor azul e 2 montanhas verdes (figura 9). Aplicado às montanhas verdes, há um algoritmo que

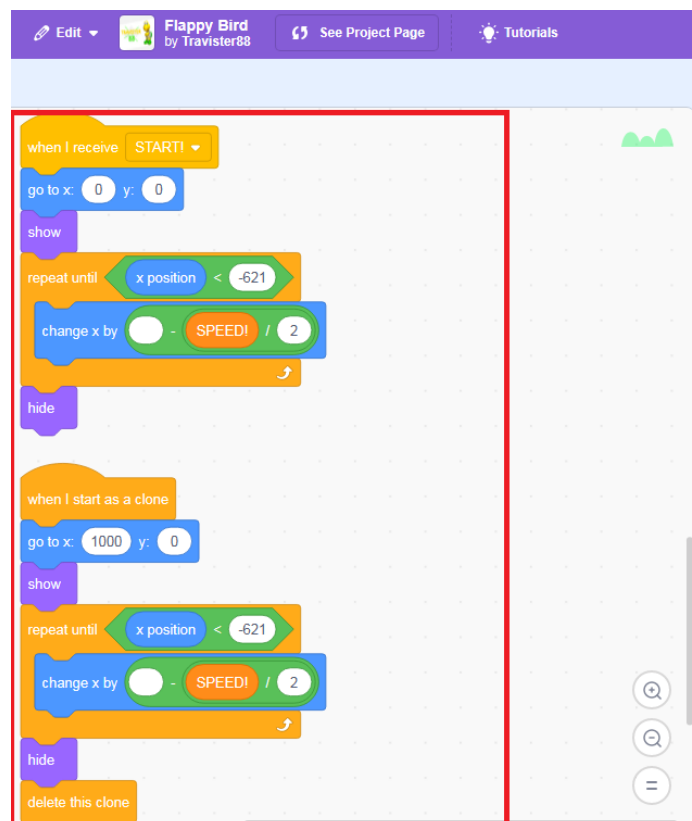
determina a movimentação destas no cenário, a fim de criar uma ilusão de avanço do personagem (figura 10).

**Figura 9** - Montanhas utilizadas no plano de fundo



Fonte: Captura de tela pelos autores, 2024.

**Figura 10** - Algoritmo de movimentação presente nas montanhas

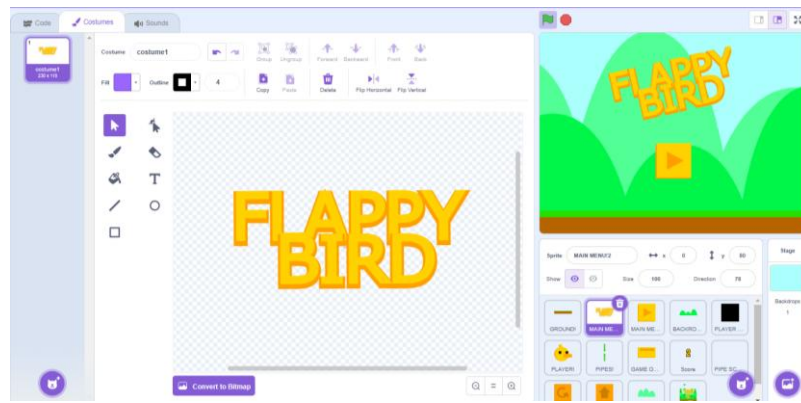


Fonte: Captura de tela pelos autores, 2024.

### 4.3 TÍTULO DO JOGO

Presente na tela de menu principal, temos o texto representando o título do jogo. Neste elemento, há um algoritmo que define um script de movimentação e rotação do texto apenas para fins de estética, sem uma interferência na jogabilidade do jogo.

**Figura 11** - Elemento "título do jogo" dentro da ferramenta de edição



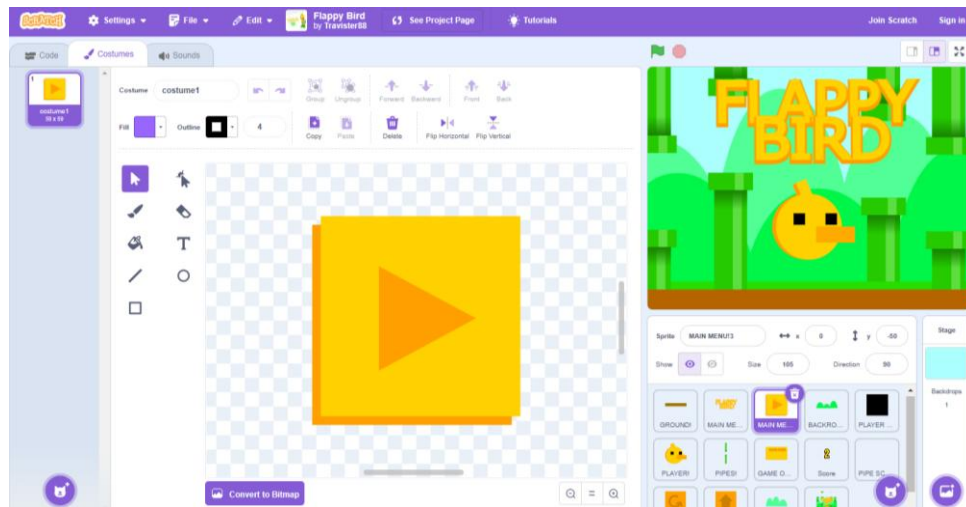
Fonte: Captura de tela pelos autores, 2024.

### 4.4 BOTÃO DE INÍCIO

Este elemento é responsável pelo início de um novo jogo, possuindo um algoritmo responsável por reiniciar o contador de obstáculos ultrapassados (figura 13).

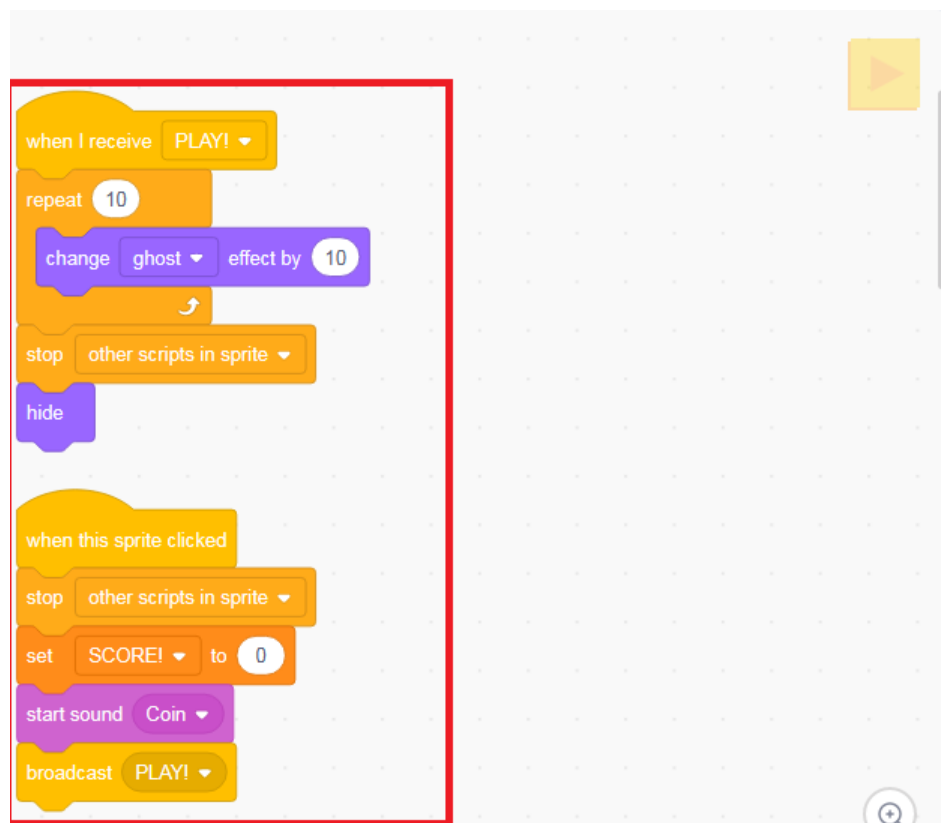


**Figura 12** - Elemento "botão de inicio" dentro da ferramenta de edição



Fonte: Captura de tela pelos autores, 2024.

**Figura 13** - Algoritmo presente no elemento "botão de inicio"



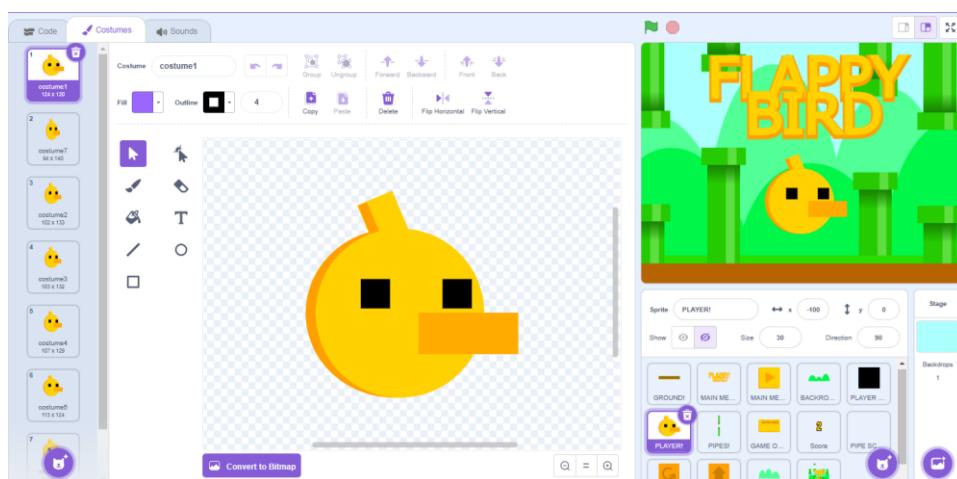
Fonte: Captura de tela pelos autores, 2024.

## 4.5 PERSONAGEM

Um dos principais elementos presentes no jogo é o personagem controlado pelo jogador. Dentro da ferramenta de edição, identificamos que o personagem possui sete variações de sprites, as quais correspondem à animação de voo acionada pelo botão de comando (figura 14). Entretanto, a lógica de controle e movimentação não está presente no código deste, já que o personagem é vinculado a uma *hitbox*.

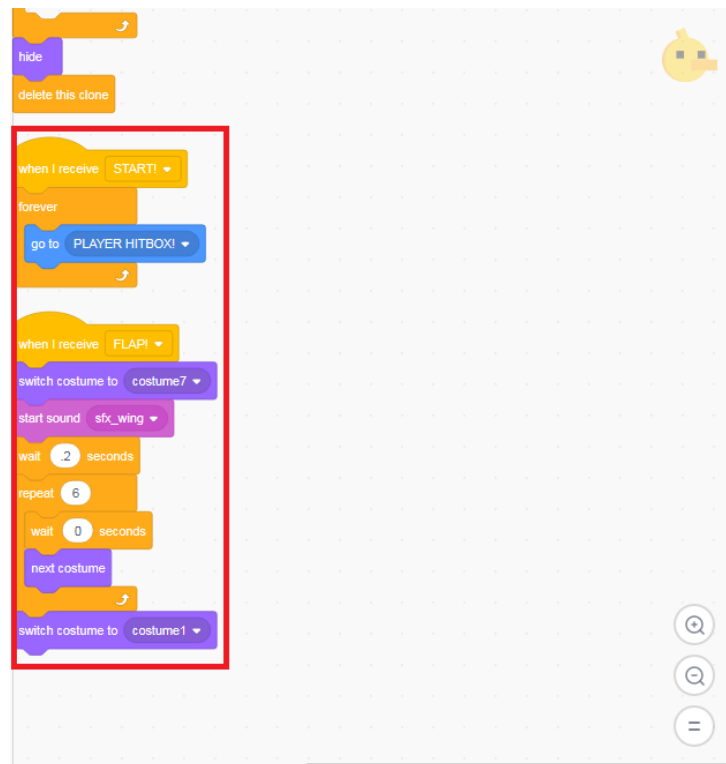
Em seu código-fonte, uma parte do algoritmo é destinada ao acionamento da animação de voo, enquanto a outra parte interrompe as operações caso o personagem colida com algum obstáculo e aciona o efeito sonoro correspondente à colisão (figuras 15 e 16).

**Figura 14** - Personagem e suas variações (à esquerda)



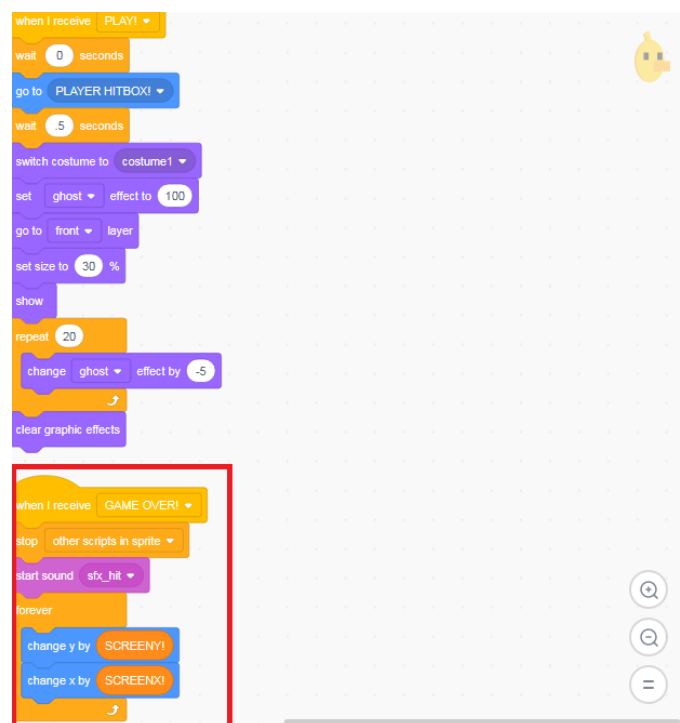
Fonte: Captura de tela pelos autores, 2024.

**Figura 15** - Algoritmo responsável pela animação de voo



Fonte: Captura de tela pelos autores, 2024.

**Figura 16** - Algoritmo que interrompe operações e aciona o efeito sonoro de colisão

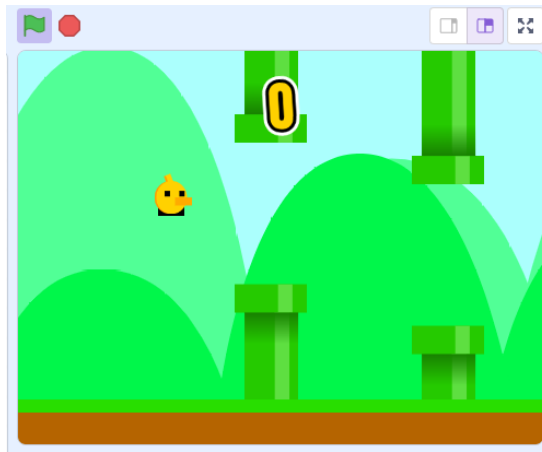


Fonte: Captura de tela pelos autores, 2024.

## 4.6 HITBOX

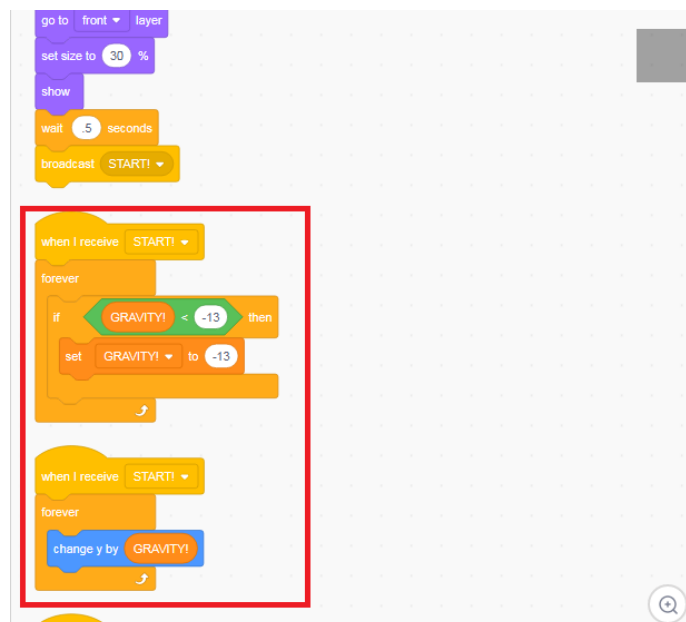
A *hitbox* é um dos elementos fundamentais em qualquer jogo. Em muitos dos casos, é ela quem determina a área de colisão e dano que qualquer personagem terá, seja ele um inimigo ou um personagem controlado pelo jogador. No contexto do jogo, a *hitbox* tem o papel de delimitar a área de colisão do personagem com os obstáculos. Através da ferramenta de edição, identificamos que a lógica de movimentação do personagem está presente em seu código (figuras 18, 19 e 21). Ela é representada por um quadrado preto que fica invisível ao jogador (figura 17).

**Figura 17** – Demonstração em jogo da *hitbox* visível



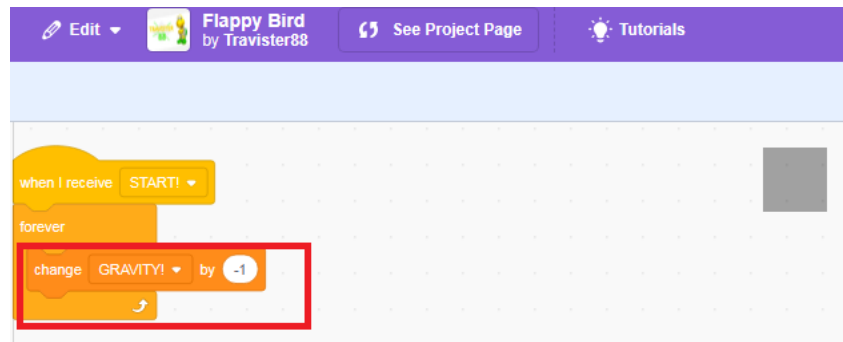
Fonte: Captura de tela pelos autores, 2024.

**Figura 18** - Algoritmo responsável por simular a gravidade do personagem



Fonte: Captura de tela pelos autores, 2024.

**Figura 19** - Algoritmo responsável por simular a gravidade do personagem



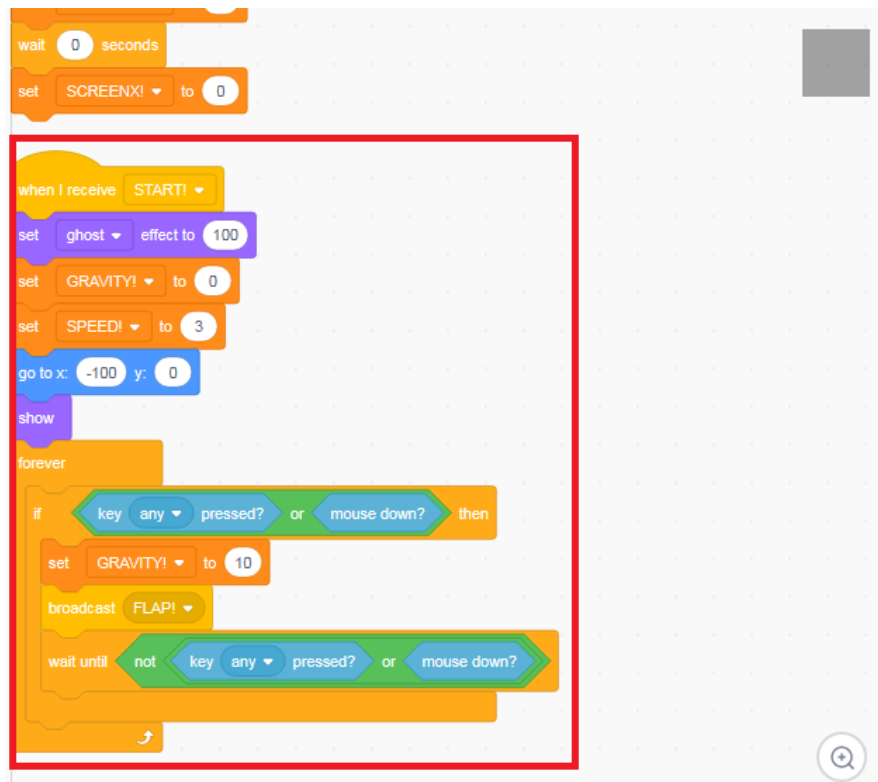
Fonte: Captura de tela pelos autores, 2024.

**Figura 20** - Algoritmo de registro dos obstáculos ultrapassados e colisão com obstáculos



Fonte: Captura de tela pelos autores, 2024.

**Figura 21** - Algoritmo responsável pelo voo do personagem



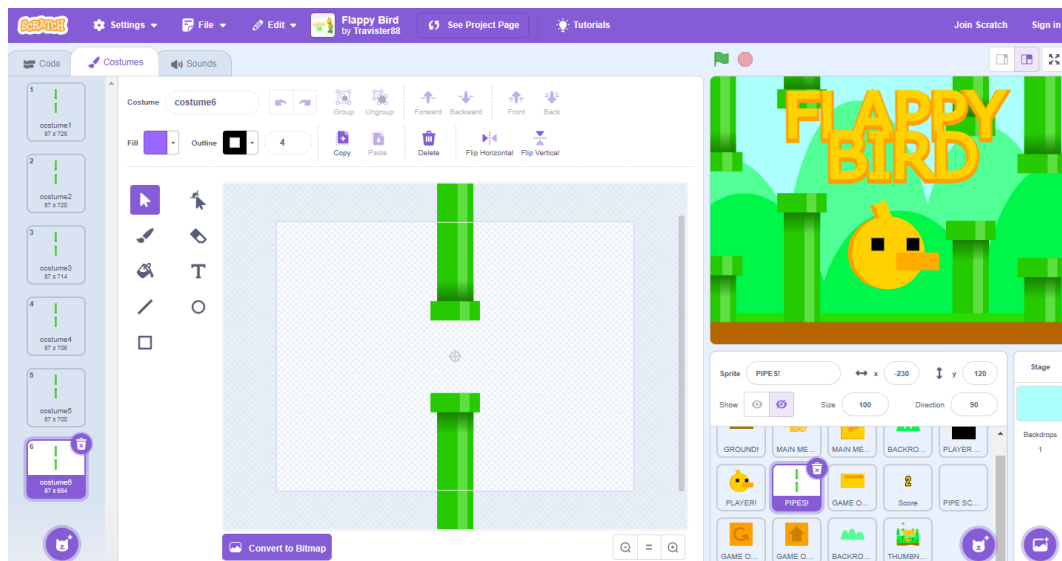
Fonte: Captura de tela pelos autores, 2024.

#### 4.7 CANOS

Os canos são os principais obstáculos enfrentados no decorrer do jogo. Através da ferramenta de edição, identificamos que há seis variações de sprites dos canos, diminuindo o espaço em que o jogador pode passar (figura 22). Em seu código-fonte, uma parte do algoritmo é responsável por criar vários clones do obstáculo (figura 24). Outra parte do algoritmo é condicionada à pontuação do jogador, fazendo com que, conforme este ultrapasse um determinado número de obstáculos, o espaço fique cada vez mais estreito, dificultando a jogabilidade (figura 25).

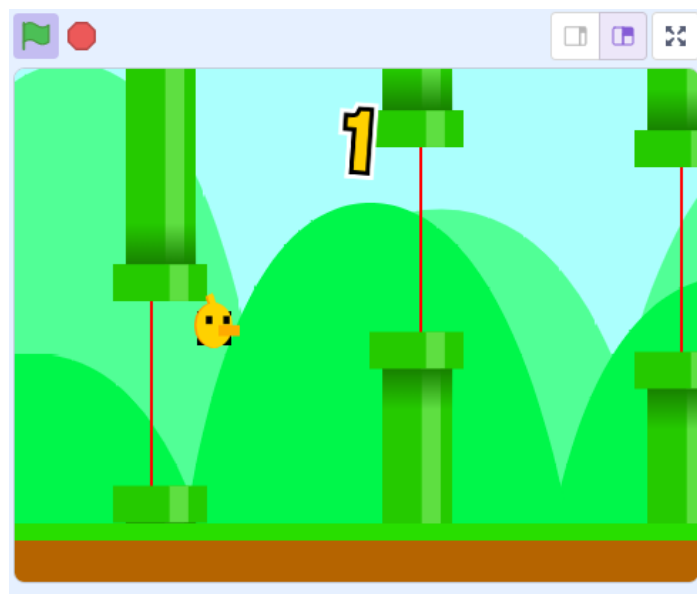
Identificamos também que, vinculadas aos canos, existem algumas linhas vermelhas, as quais são responsáveis por demarcar o ponto a ser ultrapassado pelo personagem para que seja contada a pontuação (figura 23). Estas linhas não são visíveis ao jogador durante o jogo.

**Figura 22** - Cano e suas variações (à esquerda)



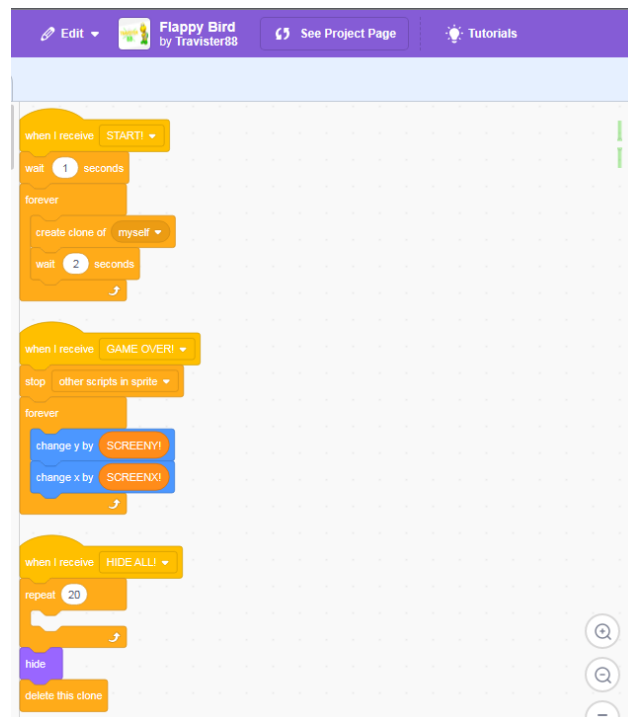
Fonte: Captura de tela pelos autores, 2024.

**Figura 23** - Demonstração em jogo das linhas vermelhas



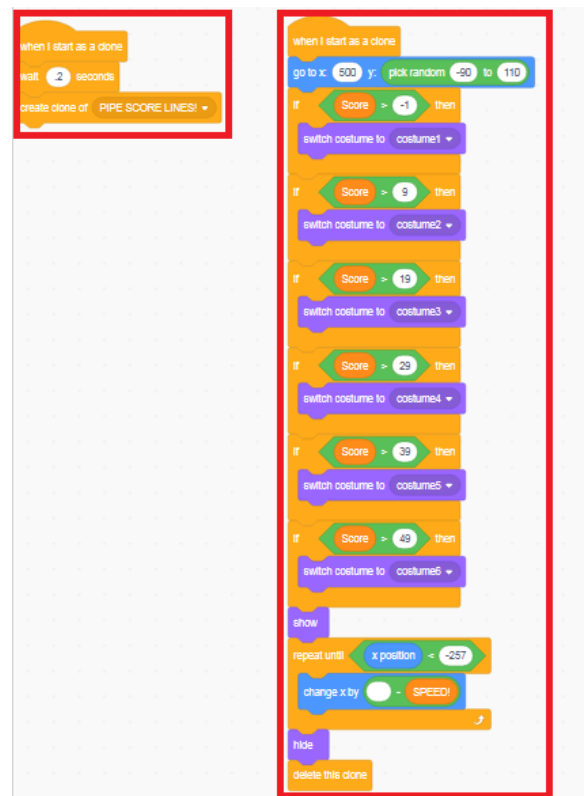
Fonte: Captura de tela pelos autores, 2024.

**Figura 24** - Algoritmo responsável por clonar obstáculos



Fonte: Captura de tela pelos autores, 2024.

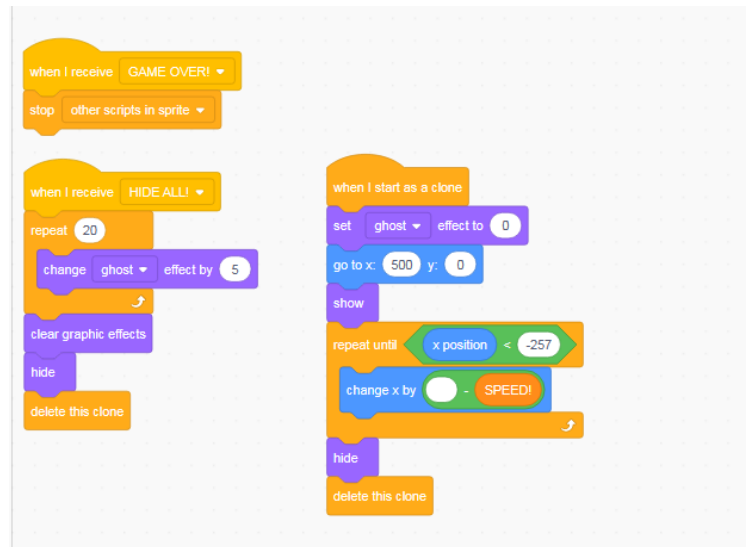
**Figura 25** - Algoritmo responsável por variar obstáculos conforme a pontuação do jogador



Fonte: Captura de tela pelos autores, 2024.



**Figura 26** - Algoritmo presente nas linhas vermelhas

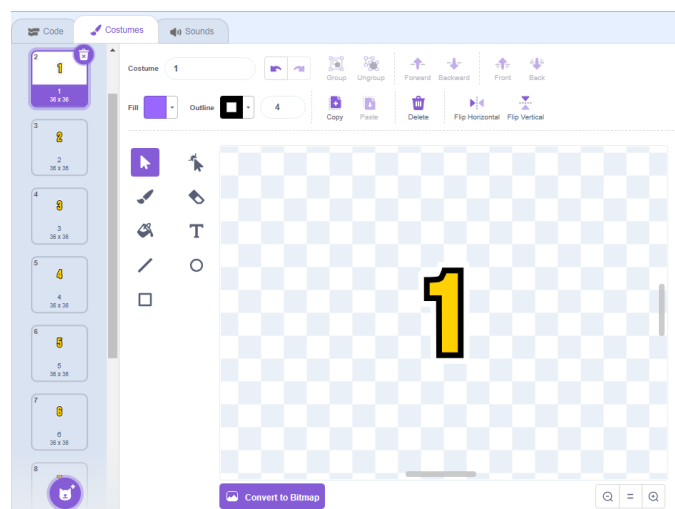


Fonte: Captura de tela pelos autores, 2024.

#### 4.8 CONTADOR DE PONTUAÇÃO

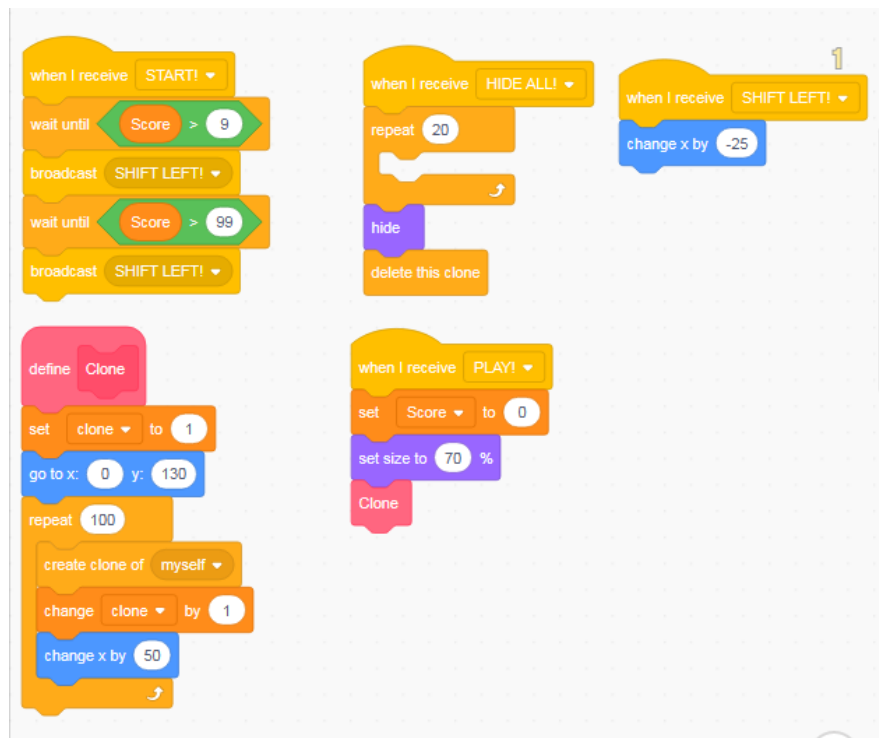
Este elemento é responsável por registrar o número de canos ultrapassados pelo jogador durante o jogo. Ele é composto por dez sprites, os quais representam os algarismos do 0 ao 9. Em seu código-fonte, o algoritmo aplicado reutiliza os algarismos caso a pontuação do jogador alcance números de duas ou três casas como, por exemplo, 10 ou 100 (figuras 28 e 29).

**Figura 27** - Contador de pontuação e suas variações (à esquerda)



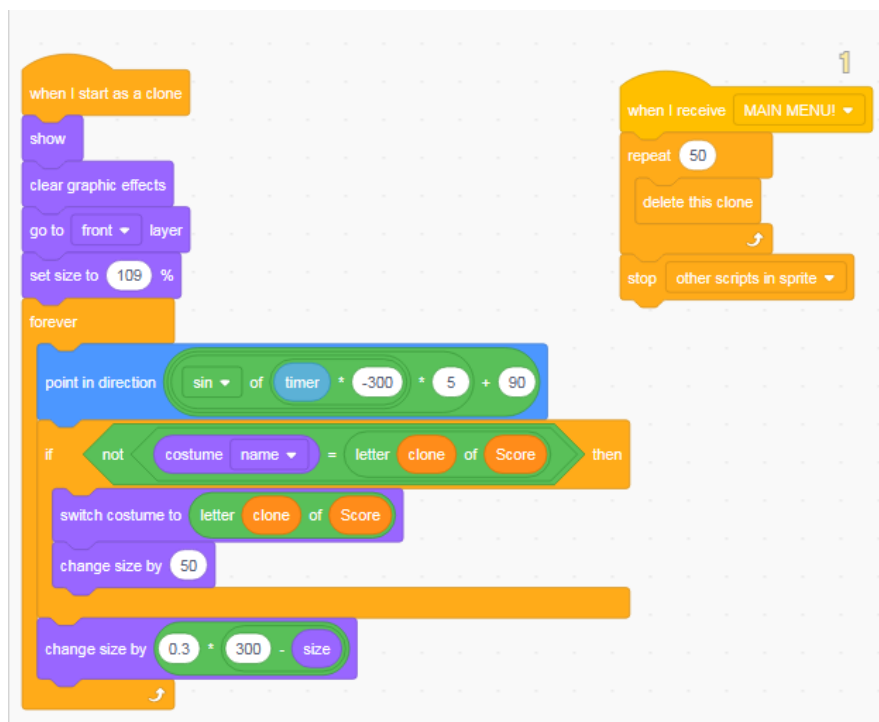
Fonte: Captura de tela pelos autores, 2024.

**Figura 28** - Algoritmo presente no contador de pontuação



Fonte: Captura de tela pelos autores, 2024.

**Figura 29** - Algoritmo presente no contador de pontuação

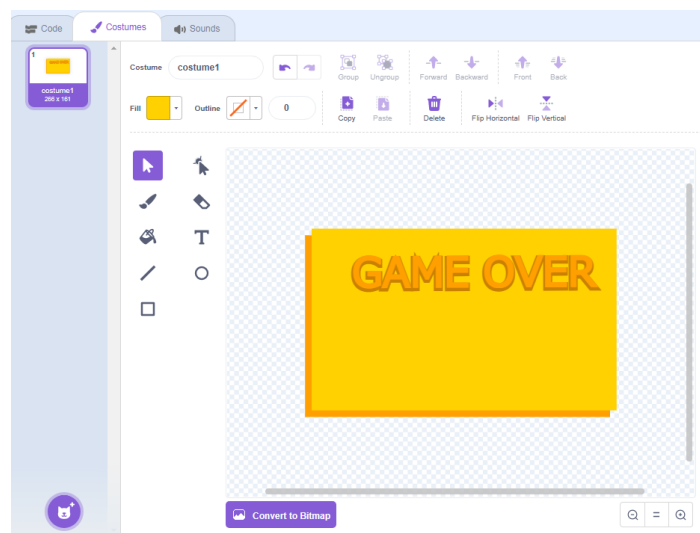


Fonte: Captura de tela pelos autores, 2024.

#### 4.9 TELA DE FIM DE JOGO

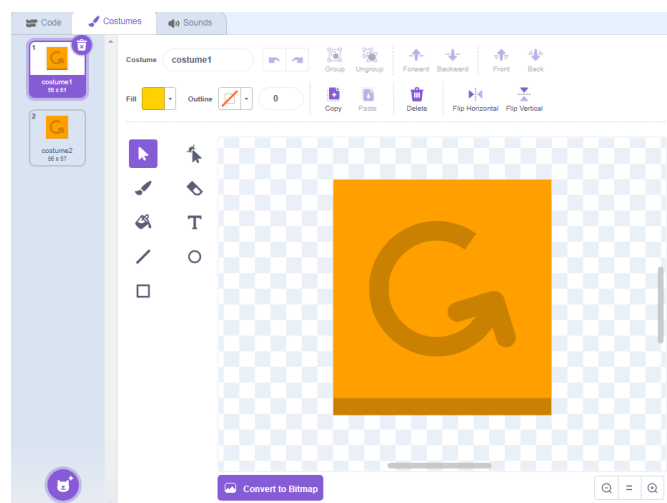
Popularmente conhecido como tela de “Game Over”, este elemento é acionado conforme a colisão do personagem com algum dos obstáculos presentes no jogo. Inclui dois botões que, conforme determinado em seus respectivos códigos-fonte, são utilizados para reiniciar um novo jogo (figura 36) e retornar ao menu principal (figura 38); também possuem um algoritmo responsável pela alternância dos sprites dos botões conforme o jogador posiciona o mouse encima destes, tendo apenas uma função estética (figuras 35 e 37).

**Figura 30** - Tela de fim de jogo dentro da ferramenta de edição



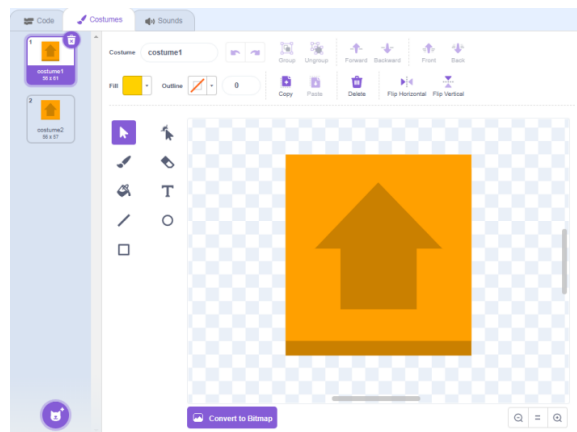
Fonte: Captura de tela pelos autores, 2024.

**Figura 31** - Botão para reiniciar um novo jogo



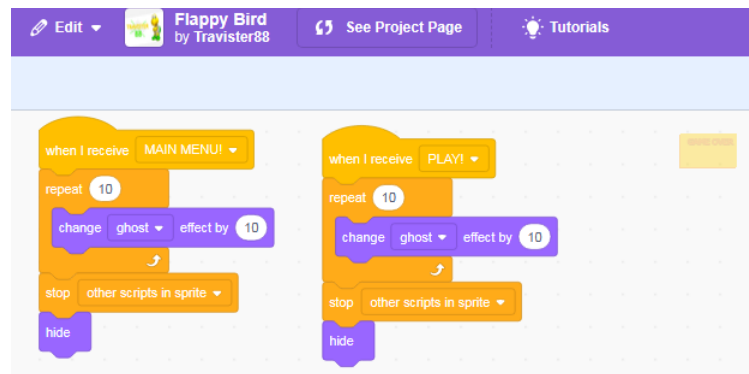
Fonte: Captura de tela pelos autores, 2024.

**Figura 32** - Botão para retornar ao menu principal



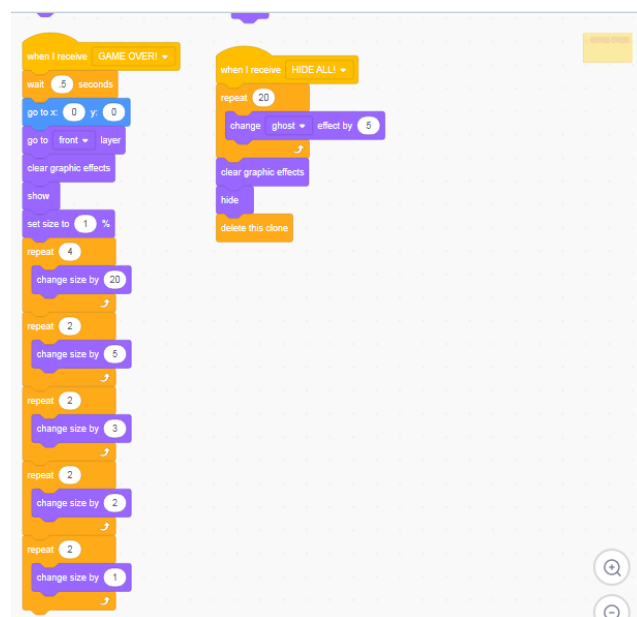
Fonte: Captura de tela pelos autores, 2024.

**Figura 33** - Algoritmo presente na tela de fim de jogo



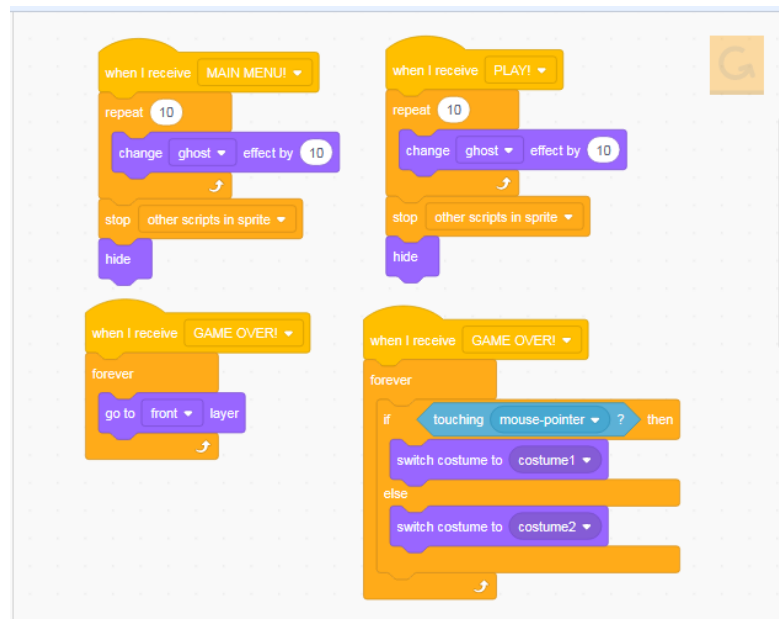
Fonte: Captura de tela pelos autores, 2024.

**Figura 34** - Algoritmo presente na tela de fim de jogo



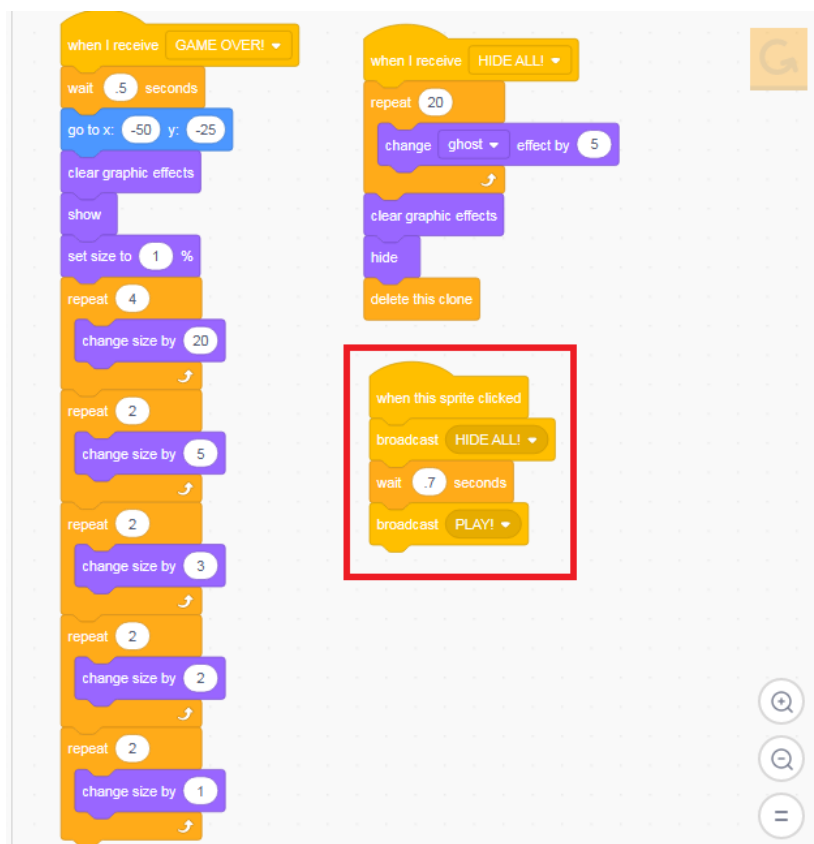
Fonte: Captura de tela pelos autores, 2024.

**Figura 35** – Algoritmo responsável por alternar sprites (botão de reiniciar jogo)



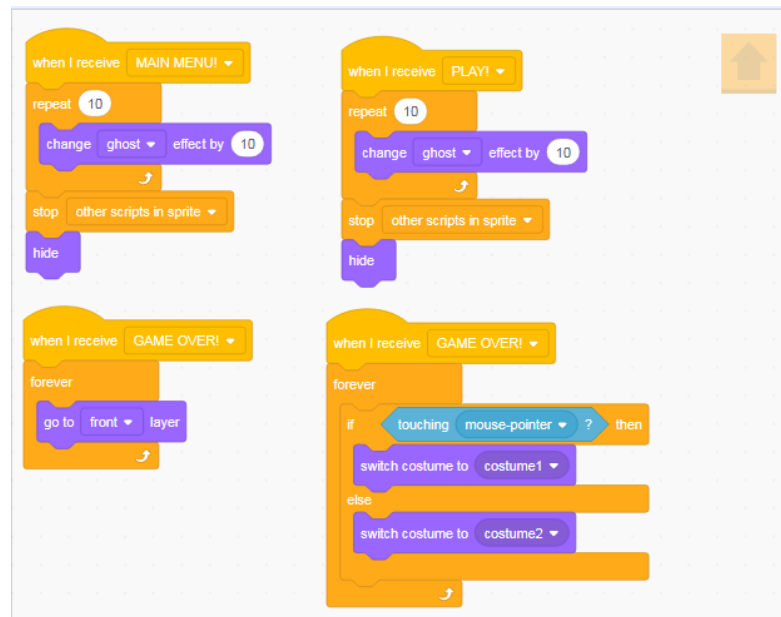
Fonte: Captura de tela pelos autores, 2024.

**Figura 36** – Algoritmo responsável por iniciar um novo jogo



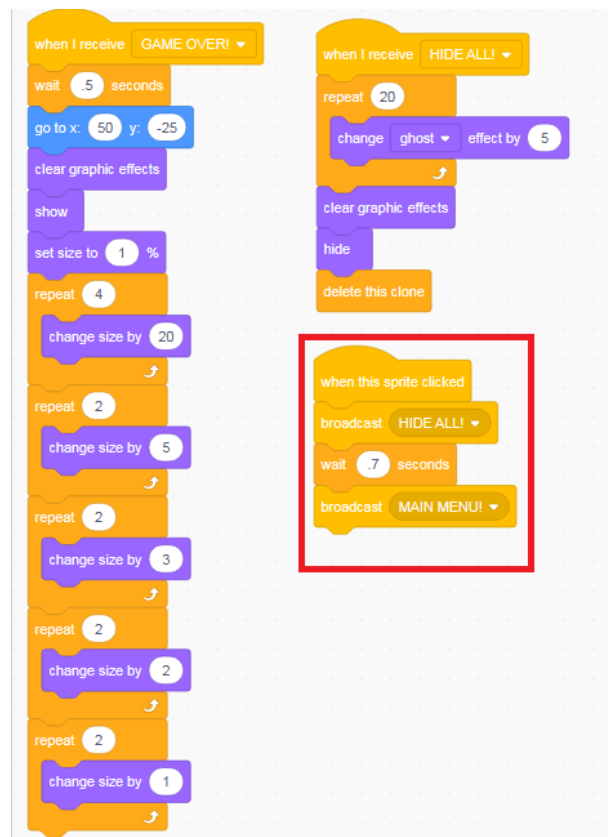
Fonte: Captura de tela pelos autores, 2024.

**Figura 37** - Algoritmo responsável por alternar sprites (botão de menu principal)



Fonte: Captura de tela pelos autores, 2024.

**Figura 38** – Algoritmo responsável por retornar ao menu principal

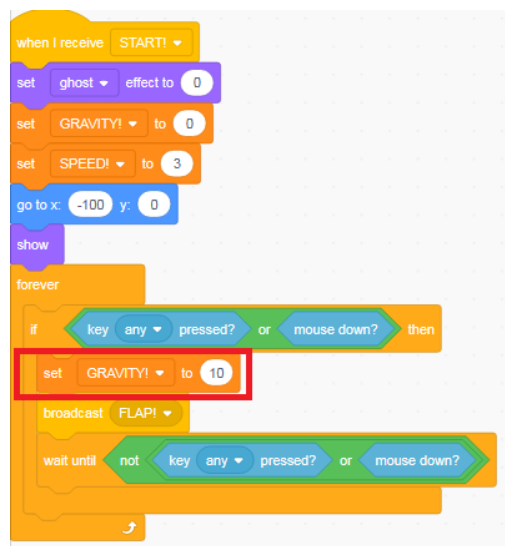


Fonte: Captura de tela pelos autores, 2024.

No que tange o teste de caixa branca, podemos determinar como possíveis sugestões de melhorias os seguintes pontos:

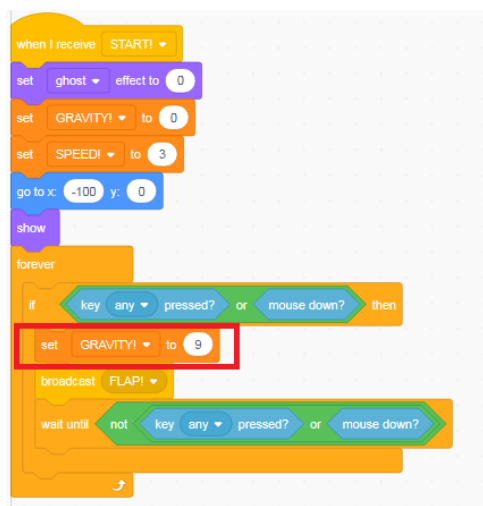
- Conforme testes por nós efetuados, identificamos que o controle do personagem se torna difícil ao passar do tempo, pois o voo feito por este é muito alto. Desta forma, consideramos o reduzir o valor da variável “GRAVITY!”, responsável pela gravidade aplicada ao acionarmos o comando de voo (figura 39). Diminuindo o valor da referida variável de 10 para 9, obtivemos uma melhora significativa na jogabilidade e desempenho.

**Figura 39** – Parte do algoritmo, originalmente com valor 10 na variável "GRAVITY!"



Fonte: Captura de tela pelos autores, 2024.

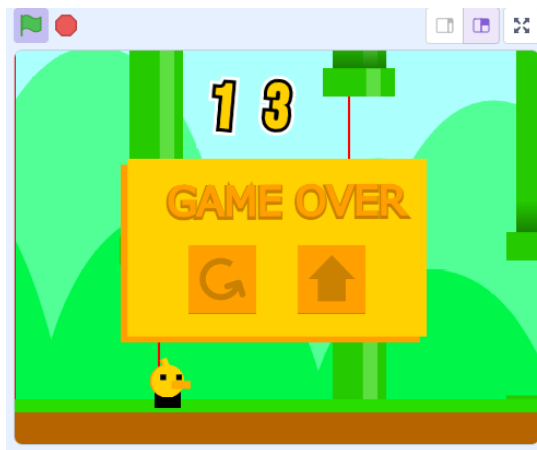
**Figura 40** - Resultado ao reduzir o valor da variável "GRAVITY!" para 9



Fonte: Captura de tela pelos autores, 2024.

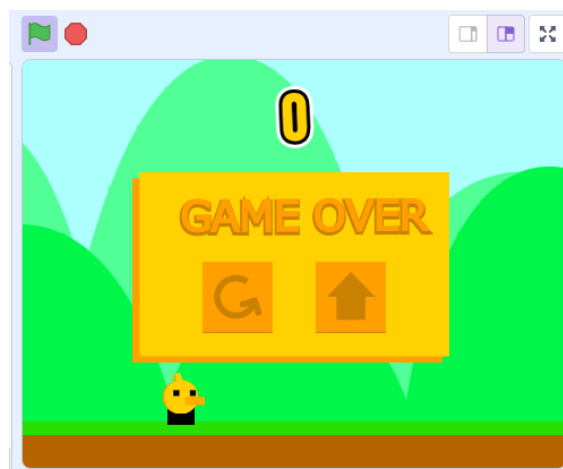
- Conforme testado e devidamente evidenciado através da ferramenta de edição, identificamos que a hitbox não está em sincronia com o sprite do personagem. Considerando que o comando de voo e a gravidade são aplicados à *hitbox*, o personagem apenas imita os comandos de movimentação enviados àquela, através do vínculo entre os elementos. Na prática, a *hitbox* acaba colidindo com um obstáculo antes mesmo do próprio sprite do personagem, resultando na derrota do jogador (figuras 41 e 42). Entretanto, em nossos testes, não foi possível sincronizar ambos os elementos.

**Figura 41** – Primeira evidência, onde é possível notar que a *hitbox* toca o chão antes do personagem



Fonte: Captura de tela pelos autores, 2024.

**Figura 42** – Segunda evidência, onde é perceptível que há um espaço entre o personagem e o chão



Fonte: Captura de tela pelos autores, 2024.



## 5 CONCLUSÃO

Após os diversos testes efetuados no jogo *Flappy Bird*, podemos dizer que todos os elementos interativos como botões estão funcionando conforme o esperado. Evidenciamos, também, que as ações e operações como o comando de voo do personagem e o contador de pontuação baseado nos obstáculos ultrapassados pelo jogador estão em perfeito funcionamento, apenas tendo sido sugeridos na amplitude da gravidade aplicada ao personagem.

No entanto, foram evidenciados alguns problemas como a falta de sincronia entre a *hitbox* e o personagem, resultado na colisão com os obstáculos existentes no jogo e ocasionando prejuízos para os jogadores que, muitas das vezes, acabam perdendo o jogo sem ao menos tocá-los. Além disso, o jogo tem alguns problemas com otimização e travamento, tanto nos computadores como nas plataformas mobile (Android e iOS).

Por fim, gostaríamos de ressaltar sobre as melhorias sugeridas por nós como: a variação dos cenários de fundo, a fim de quebrar a linearidade do jogo; a adição de um scoreboard para registro de pontuações e aumento na popularidade entre os jogadores; e correção da colisão do personagem com obstáculos para melhorar a experiência de jogo.

6 CRONOGRAMA

PERÍODO – 2024

| MESES                          | ABRIL | MAIO |
|--------------------------------|-------|------|
| Estudo e planejamento do teste | X     |      |
| Teste de software              | X     |      |
| Desenvolvimento                |       | X    |
| Entrega da versão final        |       | X    |

## REFERÊNCIAS

GASPAROTTO, Angelita Moutin Segoria; SOUZA, Karla Pires de. **A Importância da Atividade de Teste no Desenvolvimento de Software**, 2013. Disponível em: [https://abepro.org.br/biblioteca/enegep2013\\_TN\\_STO\\_177\\_007\\_23030.pdf](https://abepro.org.br/biblioteca/enegep2013_TN_STO_177_007_23030.pdf). Acesso em: 06 de maio de 2024.

SOARES, João Paulo. **Importância dos testes de software na qualidade do sistema**. TreinaWeb, [2020?]. Disponível em: <https://www.treinaweb.com.br/blog/importancia-dos-testes-de-software-na-qualidade-do-sistema>. Acesso em: 19 de abril de 2024.

Travister88. **Flappy Bird**. Scratch, 2024. Disponível em: <https://scratch.mit.edu/projects/428265264/>. Acesso em: 01 de maio de 2024.